GIẢI THÍCH LUỒNG HỆ THỐNG VÀ CÁCH TỔ CHỨC FILE CỦA 3 DỰ ÁN

Blazor Server • ASP.NET Core MVC • Razor Pages

Đề tài: Website bán hàng điện tử | Sinh viên: Dương Thạch Phú | MSSV: 3123410266

Tài liệu này tổng hợp lại **cách hệ thống hoạt động** và **cách sắp xếp file/thư mục** trong cả 3 dự án, viết theo cách dễ hiểu để dùng làm tư liệu khi viết báo cáo. Cả 3 dự án cùng làm một đề tài (website bán hàng điện tử: sản phẩm, giỏ hàng, đặt hàng, quản trị viên...), nhưng mỗi dự án dùng một công nghệ hiển thị giao diện khác nhau trong ASP.NET Core. Phần dữ liệu (cơ sở dữ liệu MySQL, Entity Framework Core, hệ thống đăng nhập/phân quyền) được tổ chức theo cùng một cách ở cả 3 dự án.

Dự án	Công nghệ giao diện	Thư mục gốc
Blazor	Blazor Server (Razor Component, SignalR)	3123410266_DuongThachPhu_Blazor
MVC	ASP.NET Core MVC (Controller + View)	3123410266_DuongThachPhu_MVC
Razor Pages	Razor Pages (Page + PageModel)	3123410266_DuongThachPhu_Razor\AspRazorPageProduct

MỤC LỤC

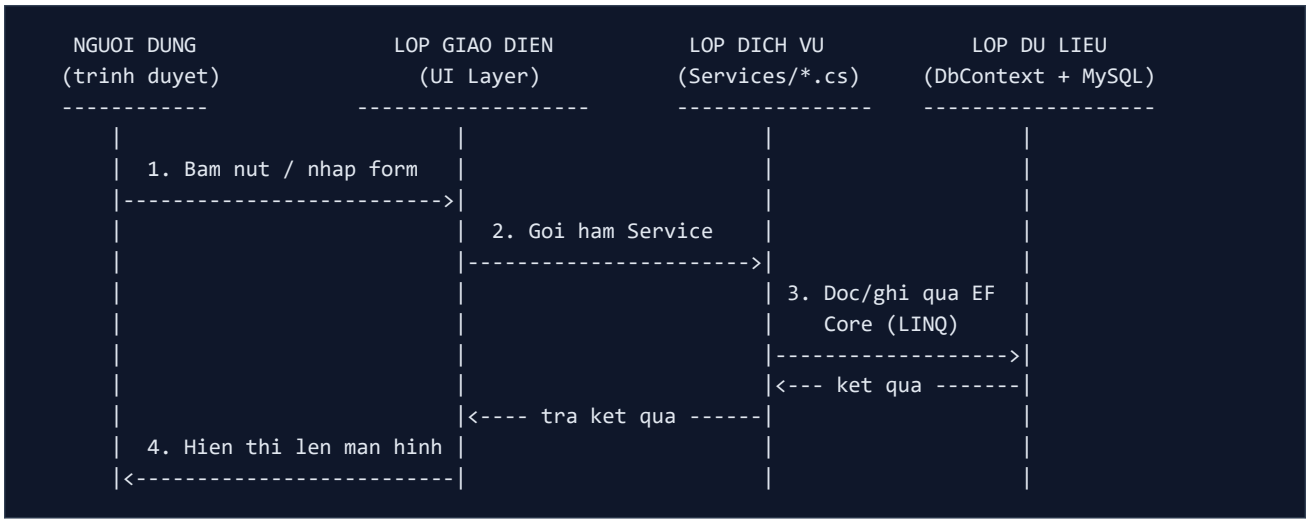
- 1. Kiến trúc chung của cả 3 dự án (mô hình phân lớp)
- 2. Hệ thống đăng nhập & phân quyền (dùng chung cho cả 3 dự án)
- 3. Luồng xử lý 1 yêu cầu — khác biệt theo từng công nghệ
- 3.1 Blazor Server 3.2 ASP.NET MVC 3.3 Razor Pages
- 4. Cấu trúc thư mục & vai trò từng file — Blazor
- 5. Cấu trúc thư mục & vai trò từng file — MVC
- 6. Cấu trúc thư mục & vai trò từng file — Razor Pages
- 7. Bảng so sánh tổng hợp 3 dự án
- 8. Minh họa 1 luồng nghiệp vụ thật qua cả 3 dự án (đặt hàng & quản lý đơn hàng)
- 9. Gợi ý khi viết báo cáo

1. KIẾN TRÚC CHUNG CỦA CẢ 3 DỰ ÁN

Dù giao diện hiển thị theo 3 cách khác nhau, cả 3 dự án đều được chia làm **3 lớp (layer)** rõ ràng. Tách lớp như vậy giúp code gọn, dễ sửa — ví dụ muốn đổi cách tính giá đơn hàng thì chỉ cần sửa 1 chỗ trong Service, không phải sửa trong từng trang giao diện.

- **Lớp giao diện (UI/Presentation):** nơi hiển thị cho người dùng và nhận thao tác (bấm nút, nhập form). Đây là phần khác nhau giữa 3 dự án: Blazor dùng Component (.razor), MVC dùng Controller + View (.cshtml), Razor Pages dùng Page + PageModel (.cshtml + .cshtml.cs).
- **Lớp dịch vụ / nghiệp vụ (Service):** nằm trong thư mục `Services/` ở cả 3 dự án. Chứa các quy tắc xử lý thật (tính tổng tiền, kiểm tra tồn kho, đổi trạng thái đơn hàng, kiểm tra quyền...). Lớp này **giống nhau gần như 100%** giữa 3 dự án — cùng tên class, cùng chức năng (ProductCatalog/ProductService, OrderService, UserService, CustomerService, RoleService, CartService).
- **Lớp dữ liệu (Data):** gồm các lớp Model (đại diện 1 dòng dữ liệu, vd Product, Order, Customer...) và 1 lớp DbContext (Entity Framework Core) chịu trách nhiệm đọc/ghi xuống cơ sở dữ liệu **MySQL**.

Sơ đồ luồng dữ liệu chung



Ví dụ: khi khách bấm "Thêm vào giỏ" ở trang sản phẩm → UI gọi `CartService` → CartService lưu/đọc giỏ hàng → khi "Đặt hàng" thì gọi tiếp `OrderService` → OrderService dùng DbContext để ghi đơn hàng mới xuống MySQL.

2. HỆ THỐNG ĐĂNG NHẬP & PHÂN QUYỀN (DÙNG CHUNG)

Cả 3 dự án đều dùng **ASP.NET Core Identity** — một bộ thư viện có sẵn của ASP.NET Core để quản lý tài khoản, mật khẩu (đã mã hoá), đăng nhập bằng cookie. Identity tự tạo ra các bảng như `AspNetUsers`, `AspNetRoles` trong MySQL.

Bên cạnh đó, đồ án tự thiết kế thêm **2 bảng phân quyền chi tiết** (không có sẵn trong Identity) để kiểm soát từng chức năng quản trị riêng biệt, thay vì chỉ có 2 mức "Admin / Khách":

- `Permission`: danh sách các "quyền" cụ thể, ví dụ quyền quản lý sản phẩm, quyền quản lý đơn hàng, quyền quản lý người dùng, quyền quản lý khách hàng...
- `RolePermission`: bảng nối, quy định "vai trò nào được phép làm việc gì" (ví dụ vai trò Admin có toàn bộ quyền, vai trò nhân viên bán hàng chỉ có quyền quản lý đơn hàng).

Khi người dùng đăng nhập thành công, hệ thống sẽ gom toàn bộ các quyền của họ lại và gắn vào **Claims** (một dạng "thẻ thông tin" đi theo người dùng trong suốt phiên đăng nhập). Mỗi khi người dùng mở một trang quản trị, hệ thống chỉ cần kiểm tra claim "permission" có chứa quyền cần thiết hay không — việc này được lập trình 1 lần duy nhất trong lớp `AppClaimsPrincipalFactory` / `AuthClaimsFactory` và dùng lại ở mọi nơi.

Tài khoản mẫu được tạo sẵn khi chạy ứng dụng lần đầu (trong `DataSeeder.cs`), giống nhau ở cả 3 dự án:

Vai trò	Email	Mật khẩu
Quản trị viên (Admin)	admin@gmail.com	admin123
Khách hàng (Customer)	user@gmail.com	admin123

3. LUỒNG XỬ LÝ 1 YÊU CẦU — KHÁC BIỆT THEO TỪNG CÔNG NGHỆ

Đây là phần khác nhau rõ nhất giữa 3 dự án: cùng là "xử lý 1 thao tác của người dùng", nhưng cách request đi từ trình duyệt vào tới code C# lại khác nhau hoàn toàn.

3.1. Blazor Server

Blazor Server **không hoạt động theo kiểu HTTP request/response truyền thống**. Sau khi trang tải lần đầu, trình duyệt mở một kết nối **SignalR (WebSocket)** sống liên tục tới server, gọi là một **"circuit"**. Mọi thao tác sau đó (bấm nút, gõ chữ, chọn dropdown...) chỉ gửi một tín hiệu nhỏ qua kết nối này, server chạy thẳng đoạn code C# nằm ngay trong file `.razor` — không cần tạo Controller hay Action riêng. Sau khi xử lý xong, Blazor tự tính phần giao diện nào thay đổi và chỉ gửi phần đó về lại trình duyệt để cập nhật (không tải lại cả trang).

```
Trinh duyet  <==== SignalR (WebSocket, giu ket noi lien tuc) ====>  Blazor Server
                                                                        |
                                                        1 file .razor = UI + code xu ly
                                                        (vd: @onclick="XoaDonHang" goi thang
                                                             ham C# trong cung file)
                                                                        |
                                                        goi truc tiep Service (cung 1 process,
                                                        KHONG di qua HTTP)
                                                                        |
                                                        Service -> StoreDbContext (EF Core) -> MySQL
                                                                        |
                                                        StateHasChanged() -> Blazor tu ve lai
                                                        phan UI thay doi -> gui ve trinh duyet
```

Đặc điểm dễ nhận ra khi đọc code: *không có Controller, không có thư mục Views — mỗi file .razor vừa là giao diện (HTML pha trộn Razor) vừa chứa code C# xử lý ngay bên dưới (khối `@code { ... }`).*

3.2. ASP.NET Core MVC

MVC hoạt động theo kiểu **HTTP truyền thống** mà hầu hết website vẫn dùng: mỗi lần người dùng bấm link hoặc submit form là một request HTTP mới được gửi lên server.

```
Trinh duyet --HTTP Request (GET/POST)--> Routing (doc URL, tim dung Controller+Action)
                                                    |
                                          Controller.Action(...) chay
                                                    |
                                          goi Service xu ly nghiep vu
                                                    |
                                          Service -> StoreDbContext (EF Core) -> MySQL
                                                    |
                                          Action chon 1 View (.cshtml) de hien thi
                                                    |
                                          View duoc render ra HTML hoan chinh
                                                    |
Trinh duyet <------------------------ tra ve HTML -----
```

Ví dụ: khi admin bấm "Hủy đơn" ở trang quản trị, trình duyệt submit một **form POST** tới action `AdminController.CancelOrder(int id)` → action gọi `OrderService.UpdateOrderStatus(...)` → sau khi xong, Controller **Redirect** trình duyệt quay lại trang danh sách (mẫu "Post → Redirect → Get", tránh việc F5 lại gửi trùng form).

3.3. Razor Pages

Razor Pages gần giống MVC (vẫn là HTTP request/response, vẫn có vòng đời tương tự), nhưng tổ chức theo **từng trang độc lập** thay vì gom nhiều action vào 1 Controller chung. Mỗi trang gồm 2 file đi kèm nhau: `Ten.cshtml` (giao diện) và `Ten.cshtml.cs` (gọi là **PageModel**, chứa các hàm `OnGet()` / `OnPost...()`).



Khác biệt chính với MVC: *MVC gom nhiều chức năng liên quan vào 1 Controller (ví dụ mọi thứ về đơn hàng admin nằm trong* `AdminController`*), còn Razor Pages tách mỗi trang thành 1 cặp file riêng theo đúng cấu trúc thư mục (*`Pages/Admin/Orders/Index.cshtml` *ứng với URL* `/Admin/Orders/Index`*) — tìm file theo URL dễ hơn, nhưng khi 1 khu vực có nhiều hành động nhỏ (nhiều handler* `OnPost...` *trong cùng 1 trang) thì file PageModel có thể khá dài.*

4. CẤU TRÚC THƯ MỤC — DỰ ÁN BLAZOR

Thư mục gốc: `3123410266_DuongThachPhu_Blazor/`

Thư mục / File	Vai trò - giải thích
`BlazorWeb.csproj`	File cấu hình project .NET: khai báo phiên bản .NET và danh sách thư viện NuGet được dùng (EF Core, MySQL, Identity...).
`Program.cs`	Nơi khởi động ứng dụng: đăng ký kết nối MySQL, đăng ký Identity (đăng nhập/phân quyền), đăng ký toàn bộ Service vào DI container, thiết lập pipeline xử lý request, seed dữ liệu mẫu khi chạy lần đầu.
`appsettings.json`	Lưu chuỗi kết nối tới MySQL và các cấu hình khác của ứng dụng.
`Components/Pages/`	Mỗi file .razor ứng với 1 trang / 1 địa chỉ (route). Gồm: Home, Products, ProductDetail, Cart (giỏ hàng), Login/Register (khách), LoginAdmin (đăng nhập quản trị — route /admin/login), Admin (toàn bộ trang quản trị, chia theo tab), OrderHistory (lịch sử đơn hàng khách), Error/NotFound.
`Components/Shared/`	Component nhỏ dùng lại nhiều nơi. OrderDetailDialog.razor: khung popup hiển thị chi tiết và tiến trình trạng thái đơn hàng, dùng cho cả khách và admin.
`Components/Layout/`	Khung bố cục bọc quanh nội dung trang. MainLayout (khung cho khách), AdminLayout (khung trang quản trị), NavMenu (menu điều hướng), ReconnectModal (thông báo khi mất kết nối SignalR — đặc thù riêng của Blazor Server).
`Data/StoreDbContext.cs`	Lớp DbContext của Entity Framework Core — khai báo các bảng (DbSet) ánh xạ xuống MySQL: Products, Orders, Customers, Users...
`Data/DataSeeder.cs`	Tạo sẵn dữ liệu mẫu khi chạy lần đầu: tài khoản admin/khách mẫu, vai trò, quyền, sản phẩm mẫu.
`Data/DesignTimeDbContextFactory.cs`	Hỗ trợ công cụ dòng lệnh EF Core (dotnet ef migrations...) tạo được DbContext lúc thiết kế; không ảnh hưởng lúc ứng dụng chạy thật.
`Migrations/`	Các file do EF Core tự sinh ra, ghi lại từng lần thay đổi cấu trúc bảng — dùng để tạo/cập nhật cấu trúc database MySQL.
`Models/`	Các lớp đại diện cho 1 dòng dữ liệu / 1 bảng trong CSDL: Product, Category, Order, OrderItem, Customer, User, Role, Permission, RolePermission...
`Services/`	Toàn bộ logic nghiệp vụ, tách khỏi giao diện: ProductCatalog, CartService, OrderService, UserService, CustomerService, RoleService; AppClaimsPrincipalFactory/AuthClaimsFactory (sinh quyền hạn - claims khi đăng nhập).
`wwwroot/`	Tài nguyên tĩnh: CSS (app.css), JavaScript, thư viện Bootstrap... trình duyệt tải trực tiếp.
`docs/`	Tài liệu thiết kế / kế hoạch phát triển trong lúc làm đồ án — không phải 1 phần của ứng dụng chạy.

Điểm đặc biệt: *dự án Blazor không có thư mục Views và không có Controller — vì giao diện và code xử lý nằm chung trong 1 file .razor.*

5. CẤU TRÚC THƯ MỤC — DỰ ÁN MVC

Thư mục gốc: `3123410266_DuongThachPhu_MVC/`

Thư mục / File	Vai trò - giải thích
`MVC.csproj / Program.cs / appsettings.json`	Vai trò giống Blazor, chỉ khác: Program.cs của MVC cấu hình thêm kiểu route cổ điển `{controller}/{action}/{id?}`.
`Controllers/`	Mỗi class xử lý nhiều hành động (action) liên quan tới nhau. AccountController (đăng nhập/đăng ký/đơn hàng của khách), AdminController (toàn bộ xử lý trang quản trị: sản phẩm, danh mục, đơn hàng, người dùng, khách hàng), CartController, HomeController, ProductsController.
`Views/`	Các file .cshtml hiển thị giao diện, đặt theo tên Controller tương ứng: Views/Account/, Views/Admin/, Views/Cart/, Views/Home/, Views/Products/.
`Views/Shared/`	Layout và partial view dùng lại ở nhiều trang: _Layout.cshtml (khung trang khách), _AdminLayout.cshtml (khung trang quản trị), _OrderStatusBadge.cshtml / _OrderStatusPipeline.cshtml (hiển thị trạng thái đơn hàng — dùng lại ở nhiều nơi để đồng bộ giao diện).
`Models/`	Giống Blazor (Product, Order, Customer, User...), có thêm `ViewModels.cs`: các lớp chỉ dùng để truyền dữ liệu giữa Controller và View (không lưu xuống CSDL), ví dụ LoginViewModel, ProductEditViewModel.
`Services/ , Data/ , Migrations/`	Vai trò giống hệt bên Blazor — cùng tên class, cùng chức năng (đây là phần dùng chung logic nhiều nhất giữa 2 dự án).
`wwwroot/`	Tài nguyên tĩnh: CSS, JS, thư viện Bootstrap/jQuery...

Điểm đặc biệt: *MVC là dự án duy nhất trong 3 dự án có khái niệm ViewModel rõ ràng và dùng SessionCart (lưu giỏ hàng vào Session của trình duyệt) thay vì lưu thẳng vào CSDL như giỏ hàng tạm.*

6. CẤU TRÚC THƯ MỤC — DỰ ÁN RAZOR PAGES

Thư mục gốc: `3123410266_DuongThachPhu_Razor/AspRazorPageProduct/`

Thư mục / File	Vai trò - giải thích
`AspRazorPageProduct.csproj / Program.cs / appsettings.json`	Vai trò giống 2 dự án trên. Riêng phần kết nối MySQL dùng thư viện Pomelo (khác MySql.EntityFrameworkCore của Blazor/MVC) nhưng cách dùng và vai trò hoàn toàn tương đương.
`Pages/ (gốc)`	Các trang dùng chung cho khách: Index (trang chủ), Details (chi tiết sản phẩm), Cart (giỏ hàng), Checkout (thanh toán/đặt hàng), OrderHistory (lịch sử đơn hàng), OrderConfirmation (xác nhận đơn hàng), Error. Mỗi trang có 2 file đi kèm: .cshtml (giao diện) + .cshtml.cs (PageModel xử lý).
`Pages/Account/`	Login, Logout, AccessDenied — xử lý đăng nhập/đăng xuất của khách.
`Pages/Admin/`	Login (đăng nhập quản trị — route /admin/login), Index (tự chuyển hướng vào Dashboard), và các thư mục con theo từng chức năng quản trị.
`Pages/Admin/Dashboard, Products, Categories, Orders, Users, Customers`	Mỗi thư mục con là 1 khu vực quản trị riêng, mỗi trang trong đó vẫn theo cặp .cshtml + .cshtml.cs như trên — đây là điểm khác rõ nhất so với MVC (MVC gom hết các chức năng quản trị vào 1 AdminController, Razor Pages tách theo từng thư mục/trang).
`Pages/Shared/`	Layout và partial dùng chung — vai trò giống Views/Shared/ của MVC: _Layout, _AdminLayout, _OrderStatusBadge, _OrderStatusPipeline.
`Services/ (+ các file I*.cs)`	Vai trò giống 2 dự án trên, nhưng mỗi Service ở đây có thêm 1 **interface** riêng đứng trước (IProductService, IOrderService, ICustomerService, IUserService, IRoleService, ICartService, IAuthService). Đây là cách tổ chức rõ ràng hơn: khi đăng ký Service vào DI, Razor Pages đăng ký theo kiểu "interface → class", còn Blazor/MVC đăng ký trực tiếp class — chức năng giống nhau, chỉ khác mức độ tách lớp trừu tượng.
`Data/AppDbContext.cs , DataSeeder.cs`	Vai trò giống StoreDbContext/DataSeeder của Blazor và MVC.
`Helpers/`	Các lớp hỗ trợ nhỏ, không thuộc Service hay Model: PaginatedList.cs (hỗ trợ phân trang danh sách dài), SessionExtensions.cs (hàm mở rộng giúp lưu/đọc dữ liệu phức tạp — như giỏ hàng — vào Session).
`Models/`	Giống 2 dự án trên, có thêm CartItem.cs (1 dòng trong giỏ hàng, lưu qua Session).
`wwwroot/`	Tài nguyên tĩnh — vai trò giống 2 dự án trên.

Điểm đặc biệt: *cấu trúc thư mục* `Pages/` *ánh xạ trực tiếp sang đường dẫn URL trên web — ví dụ file* `Pages/Admin/Orders/Index.cshtml` *chính là địa chỉ* `/Admin/Orders/Index`*, không cần khai báo route thủ công như MVC.*

7. BẢNG SO SÁNH TỔNG HỢP 3 DỰ ÁN

Tiêu chí	Blazor Server	ASP.NET MVC	Razor Pages
Nơi chứa code xử lý	Ngay trong file .razor (@code)	Controller (dùng chung nhiều action)	PageModel .cshtml.cs (riêng từng trang)
Cách giao tiếp	SignalR / WebSocket (giữ kết nối)	HTTP request/response từng lần	HTTP request/response từng lần
Có reload trang không	Không — chỉ cập nhật phần UI thay đổi	Có (hoặc Redirect sau khi xử lý)	Có (hoặc Redirect sau khi xử lý)
Định tuyến (routing)	Khai báo @page trong từng file	Theo mẫu {controller}/{action}/{id}	Theo đúng đường dẫn thư mục Pages/
Quản lý giỏ hàng	Lưu trong Service (bộ nhớ server, theo Customer)	SessionCart (Session trình duyệt)	CartService + Session (qua SessionExtensions)
Tách Service qua interface	Không (đăng ký trực tiếp class)	Không (đăng ký trực tiếp class)	Có (I...Service rồi mới tới class)
Thư mục giao diện	Components/Pages, Components/Shared, Components/Layout	Views/, Views/Shared	Pages/, Pages/Shared
Ưu điểm chính	Tương tác nhanh, thật, ít thao tác lặp	Quen thuộc, tư liệu/học liệu nhiều, dễ mở rộng Action	Tổ chức theo trang rất rõ ràng, dễ tìm file theo URL
Nhược điểm chính	Phải giữ kết nối SignalR liên tục; lỗi nếu dùng chung 1 DbContext cho nhiều luồng song song	Nhiều action gom vào 1 Controller có thể hơi dài	Nhiều trang nhỏ □ hơi nhiều file nếu chức năng phức tạp

8. MINH HỌA LUỒNG NGHIỆP VỤ THẬT QUA CẢ 3 DỰ ÁN

8.1. Luồng "Khách đặt hàng"

Bước	Blazor	MVC	Razor Pages
Xem sản phẩm, bấm thêm vào giỏ	Products.razor gọi CartService trực tiếp qua SignalR	ProductsController trả View, form POST tới CartController	Pages/Index hoặc Details gọi CartService (OnPost)
Lưu giỏ hàng	CartService (theo Customer đăng nhập)	SessionCart (Session trình duyệt)	CartService + SessionExtensions (Session)
Xác nhận đặt hàng	Cart.razor gọi OrderService.CreateOrder(...)	CartController/CheckoutAction gọi OrderService	Pages/Checkout.cshtml.cs (OnPost) gọi OrderService
Ghi xuống CSDL	StoreDbContext → MySQL	StoreDbContext → MySQL	AppDbContext → MySQL
Hiển thị kết quả	Cập nhật UI tại chỗ (không tải lại trang)	Redirect sang Cart/Success	Redirect sang Pages/OrderConfirmation

8.2. Luồng "Admin hủy đơn hàng"

Đây là ví dụ rất rõ cho thấy 3 dự án **dùng cùng 1 hàm Service** nhưng gọi từ 3 kiểu giao diện khác nhau:

```
Blazor:        Admin.razor  -->  OrderService.UpdateOrderStatus(id, "Da huy")
MVC:           AdminController.CancelOrder(id)  -->  OrderService.UpdateOrderStatus(id, "Da huy")
Razor Pages:   Pages/Admin/Orders/Index.cshtml.cs : OnPostCancelOrder(id)
                   -->  OrderService.UpdateStatus(id, "Da huy")
```

Cả 3 nơi đều gọi tới đúng 1 "hành động nghiệp vụ" giống nhau (đổi trạng thái đơn hàng sang "Đã hủy"), chỉ khác **nơi gọi** và **cách hiển thị lại kết quả** sau khi gọi — đúng như mô hình 3 lớp đã nêu ở phần 1: lớp Service không quan tâm giao diện bên trên là gì.

9. GỢI Ý KHI VIẾT BÁO CÁO

Khi viết báo cáo đồ án, có thể dùng lại nội dung và sơ đồ trong tài liệu này theo cấu trúc gợi ý sau:

- **Phần kiến trúc tổng quan:** dùng sơ đồ phân lớp ở mục 1 — nêu rõ vì sao tách 3 lớp (UI / Service / Data), nêu CSDL dùng chung là MySQL, ORM dùng chung là Entity Framework Core.
- **Phần đăng nhập/phân quyền:** trình bày theo mục 2 — giải thích Identity + 2 bảng Permission/RolePermission tự thiết kế, nêu rõ đây là phần *giống nhau hoàn toàn* ở cả 3 dự án (một điểm cộng khi báo cáo: cho thấy hiểu rõ phần lõi nghiệp vụ không phụ thuộc công nghệ hiển thị).
- **Phần so sánh công nghệ:** dùng 3 sơ đồ luồng xử lý ở mục 3 + bảng so sánh ở mục 7 — đây thường là phần được đánh giá cao vì thể hiện hiểu được *sự khác biệt bản chất* giữa Blazor Server / MVC / Razor Pages, không chỉ là "giao diện trông khác nhau".
- **Phần cấu trúc source code:** dùng 3 bảng thư mục ở mục 4-5-6, có thể chụp màn hình cây thư mục thật trong Visual Studio/VS Code để minh họa thêm.
- **Phần minh họa luồng nghiệp vụ:** dùng ví dụ ở mục 8 (đặt hàng, hủy đơn) — nên chọn thêm 1-2 đoạn code thật từ Service tương ứng để trích dẫn, vì đây là phần logic dùng chung, trích 1 lần áp dụng được cho cả 3 dự án.
- **Phần kết luận:** nêu ưu/nhược điểm từng công nghệ (dựa vào bảng mục 7) và bài học rút ra khi làm 3 dự án song song trên cùng 1 đề tài.

Tài liệu này được tổng hợp dựa trên cấu trúc source code thật của 3 dự án tại thời điểm hiện tại, dùng để hỗ trợ việc chuẩn bị báo cáo — không phải là một phần của ứng dụng đang chạy.